GUIDE D'APPRENTISSAGE · PROJET DE FORMATION

jp_morgraph

Apprendre l'IA agentique en construisant une petite salle de marché simulée, avec six agents orchestrés par LangGraph



En bref

Objectif	apprendre et pratiquer l'IA agentique, pas battre le marché
Durée	≈ 60 heures, soit 8 semaines à 7 ou 8 h par semaine, plus des bonus facultatifs
Budget	≈ 6 € en configuration économe, ≈ 13 € en configuration recommandée, 0 € en 100 % local
Équipe	6 agents IA, 2 modules de code, et toi dans le rôle du gérant
Stack	Python 3.14 · uv · LangGraph 1.2 · LangChain · LangSmith · pandas · Streamlit

Corentin Ghestem · 16 octobre 2026

Installation testée sous Python 3.14.6 le 16 octobre 2026 ; tarifs relevés le même jour. Projet strictement pédagogique : simulation uniquement, aucun conseil en investissement.

Sommaire

1. L'esprit du projet : apprendre, pas gagner . . . 3
1.1 Les compétences visées . . . 3
1.2 Mesurer la réussite autrement que par le P&L . . . 3
1.3 Ton carnet d'apprentissage . . . 4
2. Architecture . . . 4
2.1 Le cycle de décision hebdomadaire . . . 4
2.2 L'équipe . . . 4
2.3 L'état partagé . . . 5
3. Installation sous Python 3.14 . . . 6
3.1 Ce que j'ai vérifié pour Python 3.14 . . . 6
3.2 Logiciels et comptes . . . 7
3.3 Mise en place pas à pas . . . 7
3.4 Arborescence . . . 8
4. Feuille de route, étape par étape . . . 8
Phase 0 — Cadrer et ouvrir ton carnet . . . 8
Phase 1 — Environnement Python 3.14 . . . 9
Phase 2 — Premier agent : l'analyste technique . . . 9
Phase 3 — Le Data Desk . . . 9
Phase 4 — Deux analystes en parallèle . . . 10
Phase 5 — Débat Bull / Bear et trader . . . 10
Phase 6 — Risk Manager et toi en gérant . . . 10
Phase 7 — Simulation hebdomadaire . . . 11
Phase 8 — Évaluation, observabilité et démo . . . 11
5. Règles de simulation . . . 12
5.1 Conventions . . . 12
5.2 Deux biais à connaître . . . 12
6. Budget . . . 12
6.1 Tarifs au 16 octobre 2026 . . . 12
6.2 Hypothèses de consommation . . . 12
6.3 Coût selon la configuration . . . 13
6.4 Budget total du projet . . . 13
6.5 Garder la main sur les coûts . . . 13
7. Planning . . . 14
8. Pièges courants . . . 15
9. Bonus, une fois ta première release publiée . . . 15
10. Ressources et sources . . . 16
Annexe A · Squelette du graphe . . . 16
Annexe B · Modèle de prompt d'analyste . . . 18
Annexe C · Checklist de la première semaine . . . 18

1. L'esprit du projet : apprendre, pas gagner

La salle de marché n'est qu'un prétexte. Elle a l'avantage d'offrir des rôles naturels (analyste, trader, contrôle des risques), des désaccords à arbitrer et des règles à respecter : exactement ce qu'il faut pour pratiquer chaque brique de l'IA agentique. Si ta salle ne bat pas le SPY, ce n'est pas un échec ; si tu ne sais pas expliquer pourquoi elle a pris une décision, si.

1.1 Les compétences visées

Chaque phase te fait pratiquer une ou deux notions précises. La dernière colonne te donne un moyen concret de vérifier que tu les maîtrises vraiment, au-delà du « ça marche ».

Compétence	Phase	Tu la maîtrises quand…
État typé et reducers	2, 4	tu sais expliquer pourquoi `reports` a un reducer et pas `portfolio`.
Nœuds, arêtes, arêtes conditionnelles	2, 5	tu dessines le graphe de mémoire et il correspond à celui de Studio.
Outils et boucle agent ↔ outil	2	tu as codé la boucle à la main et tu la compares à `create_agent`.
Sorties structurées	2, 5	tes agents produisent 10 sorties valides sur 10.
Parallélisme avec Send	4	tu sais prédire combien de fois le nœud suivant s'exécute.
Boucles bornées	5	tu as provoqué une boucle infinie et vu `recursion_limit` l'arrêter.
Human-in-the-loop	6	tu mets le graphe en pause, modifies un ordre et reprends.
Garde-fous déterministes	6	tu prouves qu'aucun ordre exécuté ne viole une limite.
Persistance et time travel	7	tu reprends une simulation interrompue et rejoues un cycle passé.
Observabilité	1 à 8	tu retrouves dans une trace pourquoi un ordre a été refusé.
Évaluation	8	une commande relance ta batterie de scénarios après chaque changement de prompt.

1.2 Mesurer la réussite autrement que par le P&L

Le rendement reste affiché, mais seulement comme repère. Les vrais indicateurs portent sur la qualité de ton système :

Indicateur	Cible	Ce qu'il t'apprend
Sorties structurées valides	≥ 95 %	Écrire des prompts et des schémas robustes.
Décisions qui ne citent que des données présentes	100 %	Lutter contre les chiffres inventés.
Ordres exécutés hors limites	0	Encadrer des agents non déterministes par du code.
Coût par cycle (configuration économe)	≈ 0,02 $	Piloter le coût d'un système agentique.
Même snapshot, même décision	2 fois sur 3 au moins	Mesurer et réduire le non-déterminisme.
Décisions expliquées depuis la trace	Toutes	Rendre un système auditable.

1.3 Ton carnet d'apprentissage

Crée dès le premier jour un fichier `docs/journal.md`. À la fin de chaque séance, quatre lignes suffisent : ce que j'ai essayé, ce qui a cassé, ce que j'ai compris, la question qui reste ouverte. Chaque phase te propose une question pour l'alimenter. Ce carnet est le vrai livrable pédagogique du projet, et une mine d'exemples concrets pour tes entretiens.

Quatre principes qui guident tout le projet

1. Le LLM raisonne, le code calcule. Prix, indicateurs, quantités, P&L : chaque chiffre vient de Python. Les agents reçoivent des chiffres et rendent des jugements.

2. Un graphe explicite plutôt qu'un superviseur libre. Plus simple à tester, moins cher, plus facile à expliquer.

3. Zéro argent réel. Simulation uniquement ; le paper trading reste un bonus.

4. Petit, puis grand. Un agent et un titre avant six agents et trois titres.

2. Architecture

2.1 Le cycle de décision hebdomadaire

Un cycle par semaine. Le vendredi après la clôture, le graphe tourne avec les seules informations connues ce jour-là ; les ordres validés sont exécutés le lundi à l'ouverture. Un rythme hebdomadaire divise le coût par cinq par rapport au quotidien et laisse le temps de lire chaque décision.

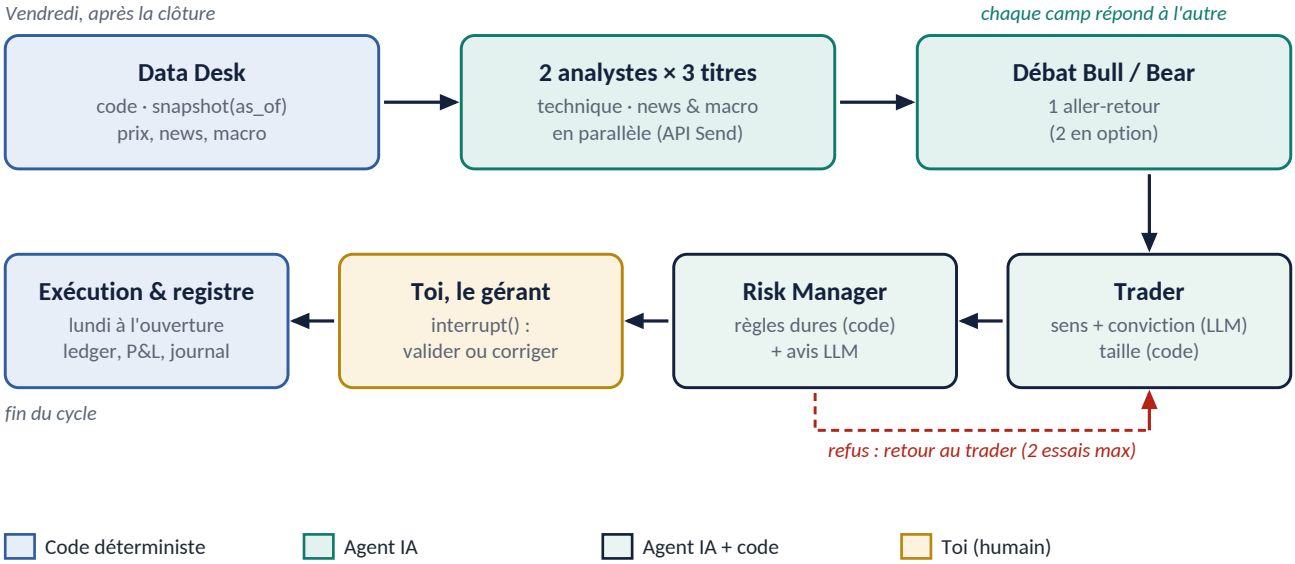


Figure 1. Un cycle de décision. Les deux analystes tournent en parallèle sur chaque titre ; le débat et le contrôle de risque sont des boucles bornées.

2.2 L'équipe

#	Membre	Type	Mission	Ce qu'il t'apprend
1	Analyste technique	Agent IA	Lire tendance, momentum et volatilité à partir des indicateurs calculés.	Outils, boucle agent ↔ outil, sortie structurée
2	Analyste news & macro	Agent IA	Résumer l'actualité de chaque titre et le contexte macro (taux, inflation, VIX).	Parallélisme (Send), reducers

#	Membre	Type	Mission	Ce qu'il t'apprend
3	Chercheur Bull	Agent IA	Défendre la thèse haussière à partir des rapports.	État partagé, boucle
4	Chercheur Bear	Agent IA	Répondre avec la thèse baissière.	Arête conditionnelle, condition d'arrêt
5	Trader	Agent IA + code	Trancher : sens et conviction par titre. Le code calcule la quantité.	Séparer jugement et calcul
6	Risk Manager	Agent IA + code	Expliquer les violations de limites et proposer un ajustement ; seules les règles en code décident.	Garde-fous, boucle de renvoi
—	Data Desk	Code	Télécharger, mettre en cache, calculer les indicateurs, produire `snapshot(as_of)`.	Discipline des données
—	Exécution & registre	Code	Simuler l'exécution, tenir cash et positions, calculer la valeur liquidative, journaliser.	Tests unitaires
—	Toi, le gérant	Humain	Valider, refuser ou corriger les ordres pendant une pause du graphe.	Human-in-the-loop

> **Et si tu veux seulement 5 agents ?**
> Fusionne les deux analystes en un seul agent doté de tous les outils. Tu économises environ 20 % de tokens, mais tu perds l'exercice de parallélisme de la phase 4 : c'est pourquoi ce guide conseille de garder les six.

2.3 L'état partagé

Tous les nœuds lisent et écrivent le même état. Les champs alimentés en parallèle (les rapports, le débat) ont un *reducer* `operator.add` qui additionne les contributions au lieu de les écraser.

Schéma d'état de départ

```
# src/jp_morgraph/state.py
from operator import add
from typing import Annotated, Literal
from pydantic import BaseModel, Field
from typing_extensions import TypedDict

class AnalystReport(BaseModel):
    analyst: Literal["technique", "news_macro"]
    ticker: str
    signal: Literal["achat", "neutre", "vente"]
    conviction: float = Field(ge=0, le=1)
    arguments: list[str] = Field(max_length=5)

class OrderProposal(BaseModel):
    ticker: str
    side: Literal["buy", "sell", "hold"]
    conviction: float = Field(ge=0, le=1)
    quantity: int = 0                   # remplie par le code, jamais par le LLM
    rationale: str

class RiskVerdict(BaseModel):
    approved: bool
    breaches: list[str]                 # règles violées, détectées par le code
    comment: str                        # explication du Risk Manager
```

```
class DeskState(TypedDict, total=False):
    as_of: str                                      # vendredi de décision (AAAA-MM-JJ)
    universe: list[str]                             # 3 titres
    snapshot: dict                                  # résumé produit par le Data Desk
    portfolio: dict                                 # cash, positions, valeur liquidative
    reports: Annotated[list[AnalystReport], add]    # écrits en parallèle
    debate: Annotated[list[str], add]
    debate_round: int
    orders: list[OrderProposal]
    risk: RiskVerdict
    risk_retries: int
    auto_approve: bool                              # True en simulation, False en mode interactif
    approved_orders: list[OrderProposal]
    fills: list[dict]
```

3. Installation sous Python 3.14

3.1 Ce que j'ai vérifié pour Python 3.14

Le 16 octobre 2026, j'ai installé Python 3.14.6 avec uv, ajouté toutes les dépendances de ce guide, puis exécuté le squelette de l'annexe A avec des nœuds factices (parallélisme, débat, renvoi au trader, pause humaine, reprise depuis SQLite) ainsi que `langgraph dev`. Tout fonctionne.

Paquet	Version testée	Remarque
langgraph	1.2.15	Ses métadonnées PyPI ne citent que Python 3.10 à 3.13, mais il s'installe et fonctionne sous 3.14.
langchain / langchain-core	1.4.6 / 1.6.12	Python 3.14 officiellement déclaré.
langchain-anthropic	1.7.6	Python 3.14 officiellement déclaré.
langgraph-checkpoint-sqlite	3.1.2	Testé : sauvegarde, pause et reprise.
langgraph-cli[inmem]	0.4.36	`langgraph dev` démarre et sert LangGraph Studio.
pydantic	2.14.1	L'ancien avertissement « Pydantic V1 » n'apparaît plus.
pandas, pyarrow, yfinance	3.0.6, 26.0.0, 1.7.0	Roues disponibles pour 3.14.
streamlit, finnhub-python, fredapi	versions du jour	Installés sans erreur.

Trois points à connaître sous 3.14

Avertissement inoffensif. langsmith peut afficher un `DeprecationWarning` sur `asyncio.iscoroutinefunction` : il n'a aucun effet sur le projet.

Avertissement de désérialisation. Quand l'état contient tes propres classes Pydantic et que tu utilises un checkpointer, LangGraph signale des « types non enregistrés ». La parade, montrée en annexe A, consiste à déclarer ces classes au sérialiseur. C'est aussi une bonne occasion de comprendre ce qu'est un checkpoint.

Plan B. Si un paquet ajouté plus tard n'avait pas de version pour 3.14, `uv python pin 3.13` suivi de `uv sync` suffit : le reste du guide est identique.

3.2 Logiciels et comptes

Élément	Usage	Statut	Coût
Python 3.14 (via uv)	Langage du projet	Obligatoire	Gratuit
uv (Astral)	Python, environnement virtuel, dépendances, lockfile	Obligatoire	Gratuit
Git, GitHub, VS Code	Versioning, vitrine, édition	Obligatoire	Gratuit
Fournisseur LLM : Anthropic, OpenAI ou Mistral	Le cerveau des agents ; un seul suffit	Obligatoire	Paiement à l'usage
LangSmith (plan Developer)	Traces, évaluations, Studio ; 1 siège, 5 000 traces de base par mois	Recommandé	Gratuit
Finnhub	News d'entreprise ; environ un an d'historique et ≈ 60 appels/min en gratuit	Recommandé	Gratuit
FRED	Séries macro	Recommandé	Gratuit
Ollama	Modèles locaux, option zéro euro	Optionnel	Gratuit
Docker, Alpaca paper	Uniquement pour les bonus	Optionnel	Gratuit

Avant le premier appel payant

Fixe un plafond de dépenses mensuel dans la console de ton fournisseur (5 à 10 € suffisent pour ce projet) et crée une clé dédiée. Les nouveaux comptes Anthropic reçoivent un petit crédit d'essai qui couvre déjà les premières phases.

3.3 Mise en place pas à pas

Terminal

```
# 1. Installer uv (macOS / Linux)
curl -LsSf https://astral.sh/uv/install.sh | sh
# 1 bis. Installer uv (Windows, PowerShell)
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"

# 2. Dossier local hors iCloud (jamais Bureau ni Documents), puis Python et squelette
mkdir -p ~/dev && cd ~/dev
uv python install 3.14
uv init jp_morgraph --package --python 3.14 && cd jp_morgraph && git init

# 3. Le cœur agentique
uv add langgraph langchain langchain-anthropic langgraph-checkpoint-sqlite \
       pydantic python-dotenv langsmith

# 4. Les données et le dashboard
uv add pandas pyarrow yfinance finnhub-python fredapi streamlit plotly

# 5. Les outils de développement
uv add --dev pytest ruff ipykernel "langgraph-cli[inmem]"

# 6. Selon tes choix
uv add langchain-openai                                  # si tu prends OpenAI
uv add langchain-mistralai                               # si tu prends Mistral
uv add langchain-ollama                                  # modèles locaux
```

Ton `pyproject.toml` contiendra `requires-python = ">=3.14"`. Les secrets vont dans un `.env` jamais commité ; versionne à la place un `.env.example` vide.

Sur Mac, reste hors d'iCloud. Si « Bureau et Documents » est synchronisé, ne crée pas le projet dedans : .venv, .git et data/ regroupent des milliers de petits fichiers qu'iCloud synchronise ou décharge (« Optimiser le stockage »), au risque de conflits et de fichiers corrompus. Travaille dans ~/dev ; GitHub sert de sauvegarde.

.env.example

```
# Un seul fournisseur suffit : ANTHROPIC_API_KEY, OPENAI_API_KEY ou MISTRAL_API_KEY
ANTHROPIC_API_KEY=
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=
LANGSMITH_PROJECT=jp_morgraph
FINNHUB_API_KEY=
FRED_API_KEY=
```

Pour LangGraph Studio, déclare ton graphe dans `langgraph.json` puis lance `uv run langgraph dev`.

langgraph.json

```
{
  "dependencies": ["."],
  "graphs": { "desk": "./src/jp_morgraph/graph.py:graph" },
  "env": ".env"
}
```

3.4 Arborescence

```
jp_morgraph/
├── pyproject.toml, uv.lock, langgraph.json, .env.example
├── docs/
│   ├── cadrage.md
│   └── journal.md            # ton carnet d'apprentissage
├── data/                     # cache Parquet (ignoré par git)
├── src/jp_morgraph/
│   ├── config.py             # titres, limites de risque, modèle par agent
│   ├── data.py               # téléchargement, cache, indicateurs, snapshot(as_of)
│   ├── tools.py              # outils @tool des analystes
│   ├── agents/               # un fichier par agent, avec son prompt
│   ├── desk.py               # règles de risque, exécution simulée, registre
│   ├── state.py
│   ├── graph.py
│   └── simulate.py           # modes replay et prospectif, rapport
├── app/dashboard.py          # Streamlit
├── evals/                    # scénarios figés et évaluateurs
└── tests/                    # pytest, sans réseau ni appel payant
```

4. Feuille de route, étape par étape

Neuf phases courtes. Chacune présente un objectif, des étapes, un exercice « pour vraiment comprendre » qui consiste souvent à casser volontairement quelque chose, une question pour ton carnet et un critère de fin.

PHASE 0 **Cadrer et ouvrir ton carnet** Semaine 1 ≈ 2 h

1. Rédige `docs/cadrage.md` en une demi-page. Objectif : apprendre l'IA agentique. Périmètre : 3 actions et le SPY, un cycle par semaine, achats uniquement, 100 000 $ fictifs. Hors périmètre : argent réel, haute fréquence, optimisation du rendement.
2. Choisis trois actions américaines de secteurs différents (par exemple une technologique, une banque et une pétrolière) pour que tes analystes aient des histoires différentes à raconter.
3. Fixe les limites de risque : 40 % maximum par ligne, exposition totale ≤ 100 %, pas de levier, cash jamais négatif.
4. Crée `docs/journal.md` et le dépôt GitHub.

Critère de fin Cadrage et carnet commités.

PHASE 1 Environnement Python 3.14

Semaine 1 ≈ 4 h

1. Installe uv et Python 3.14, crée le projet et ajoute les dépendances (§ 3.3).
2. Crée tes comptes, remplis le `.env` et fixe ton plafond de dépenses.
3. **Hello LLM** : appelle `init_chat_model("anthropic:claude-haiku-5-5")` et affiche la réponse.
4. **Hello LangGraph** : un graphe de deux nœuds avec un état TypedDict.
5. Ouvre la trace dans LangSmith, puis le graphe dans Studio avec `uv run langgraph dev`.

Pour ton carnet — que contient exactement une trace, et combien a coûté ton Hello LLM ?

Critère de fin `uv run pytest` passe, une trace est visible, le graphe s'affiche dans Studio.

PHASE 2 Premier agent : l'analyste technique

Semaine 2 ≈ 8 h

Objectif : maîtriser les briques fondamentales sur un seul agent et un seul titre.

1. Suis le cours gratuit « Introduction to LangGraph » de LangChain Academy, au moins les modules sur l'état, les arêtes conditionnelles et les outils.
2. Écris deux outils, `get_price_history` et `compute_indicators`, d'abord sur des données factices.
3. Construis l'agent **à la main** : un nœud LLM, un `ToolNode` et une arête conditionnelle « appel d'outil ou fin ». C'est le meilleur moyen de comprendre la boucle agent ↔ outil.
4. Termine par une sortie structurée `AnalystReport` avec `with_structured_output`.
5. Lance-le dix fois et corrige le prompt jusqu'à obtenir dix sorties valides.

Pour vraiment comprendre — refais le même agent avec `create_agent` de LangChain et compare les deux traces : tu verras ce que la version préfabriquée fait pour toi.

Pour ton carnet — que se passe-t-il quand un outil lève une exception, et où le vois-tu dans la trace ?

Critère de fin 10 sorties valides sur 10, et tu expliques la boucle agent ↔ outil sans regarder le code.

PHASE 3 Le Data Desk

Semaine 3 ≈ 6 h

Objectif : des données fiables, en cache, sans fuite du futur.

1. Télécharge en une fois trois ans de prix hebdomadaires ajustés pour tes 3 titres et le SPY avec yfinance, puis garde-les en Parquet dans `data/`.
2. Calcule les indicateurs en pandas : rendements à 4 et 12 semaines, moyennes mobiles 10 et 40 semaines, RSI, volatilité, drawdown.
3. Récupère les news Finnhub (environ un an d'historique en offre gratuite) et garde-les en cache : ton historique s'allongera au fil des semaines.
4. Ajoute la macro FRED (`DGS10`, `CPIAUCSL`, `UNRATE`, `VIXCLS`) en respectant les dates de publication : un chiffre mensuel n'est connu qu'une fois publié.
5. Écris `snapshot(as_of)` : un résumé d'environ 300 tokens par titre, limité à ce qui était connu à la date.
6. Teste l'absence de fuite : pour 20 dates tirées au hasard, aucune donnée du snapshot n'est postérieure à `as_of`.

Piège — yfinance renvoie des erreurs 429 « Too Many Requests » quand on enchaîne les requêtes ; d'où le téléchargement groupé et le cache.

Pour ton carnet — quelle donnée a été la plus difficile à dater correctement ?

Critère de fin snapshot reproductible, tests verts, aucune requête réseau pendant une simulation.

PHASE 4 Deux analystes en parallèle

Semaine 4 ≈ 6 h

Objectif : 2 analystes × 3 titres = 6 rapports produits en parallèle.

1. Crée l'analyste news & macro, avec son prompt et son format de sortie.
2. Utilise l'API `Send` pour lancer un même nœud `analyst` pour chaque couple (titre, spécialité).
3. Déclare le reducer `operator.add` sur `reports`.
4. Mesure tokens, coût et latence par cycle dans LangSmith.

Pour vraiment comprendre — retire le reducer et relance : LangGraph lève `InvalidUpdateError` parce que six nœuds écrivent la même clé dans le même pas. Remets-le et explique pourquoi l'erreur disparaît.

Pour ton carnet — combien de fois le nœud suivant s'exécute-t-il après un fan-out, et pourquoi ?

Critère de fin 6 rapports valides par cycle, pour environ 0,01 $ en petit modèle.

PHASE 5 Débat Bull / Bear et trader

Semaine 5 ≈ 8 h

Objectif : transformer six avis en décisions argumentées.

1. Bull lit les rapports et défend la hausse ; Bear lui répond. Un aller-retour, borné par un compteur `debate_round` et une arête conditionnelle. Passe à deux allers-retours quand tout fonctionne.
2. Le Trader tranche pour chaque titre : sens (acheter, vendre, conserver) et conviction entre 0 et 1, en sortie structurée `OrderProposal`.
3. Le code calcule la quantité : poids cible = conviction × 40 % de la valeur du portefeuille.

Pour vraiment comprendre — supprime la condition d'arrêt du débat et observe `GraphRecursionError` interrompre le graphe.

Pour ton carnet — le débat change-t-il vraiment la décision ? Compare sur cinq semaines avec un trader qui ne lit que les rapports : c'est ta première expérience d'ablation.

Critère de fin chaque ordre est relié à ses arguments, et l'effet du débat est mesuré.

PHASE 6 Risk Manager et toi en gérant

Semaine 6 ≈ 7 h

Objectif : la salle ne peut pas violer ses limites, et tu gardes le dernier mot.

1. Écris les règles dures en Python : 40 % par ligne, exposition ≤ 100 %, cash ≥ 0, pas de vente à découvert.
2. Le Risk Manager explique en langage clair les violations détectées par le code et propose un ajustement ; il ne décide jamais seul.
3. En cas de refus, retour au Trader avec le motif, deux essais au maximum.
4. Avec `interrupt()`, le graphe s'arrête et te présente les ordres ; tu reprends avec `Command(resume=...)`.

5. Écris l'exécution et le registre : exécution le lundi à l'ouverture, slippage de 5 points de base, frais de 0,05 %, suivi du cash, des positions et de la valeur liquidative, journal JSONL.

Pour vraiment comprendre — pendant la pause, corrige la quantité d'un ordre au lieu de répondre oui ou non, puis vérifie dans l'état que ta correction a bien été appliquée.

Pour ton carnet — pourquoi ne pas laisser le LLM vérifier seul le respect des limites ?

Critère de fin aucun ordre exécuté hors limites sur 52 cycles, tests du registre verts.

PHASE 7 Simulation hebdomadaire

Semaine 7
≈ 8 h

1. **Mode replay** : boucle sur les 52 derniers vendredis, avec `auto_approve=True`.
2. **Persistance** : un checkpointer SQLite et un `thread_id` par cycle. Arrête le processus en plein milieu et reprends là où il s'est arrêté.
3. **Mode prospectif** : chaque vendredi à venir, un cycle en conditions réelles avec ta validation. C'est le seul test qui échappe au biais de mémoire du modèle (§ 5.2).
4. **Rapport** : courbe de valeur liquidative face au SPY (un repère, pas un objectif), indicateurs du § 1.2, coût total.

Pour vraiment comprendre — avec `get_state_history`, remonte au checkpoint d'un cycle passé, modifie un prompt et rejoue depuis ce point : c'est le « time travel » de LangGraph.

Pour ton carnet — qu'est-ce qui, dans les traces, explique la pire décision de l'année ?

Critère de fin un rapport généré par une seule commande, et une reprise après interruption démontrée.

PHASE 8 Évaluation, observabilité et démo

Semaine 8
≈ 9 h

1. Constitue 15 scénarios figés avec un comportement attendu. Par exemple : limite violée → ordre refusé ; news très négative → pas d'achat fort ; donnée manquante → l'analyste le signale.
2. Écris deux familles d'évaluateurs : déterministes (schéma, limites) et LLM-as-judge (fidélité aux données, qualité des arguments, notée de 1 à 5).
3. Compare deux configurations sur cette batterie, tout en petit modèle puis Trader en Sonnet 5.5 : qualité, coût, latence.
4. Monte un dashboard Streamlit minimal : valeur liquidative, positions, dernier débat, coûts.
5. Rédige le README : schéma, démarrage rapide, ce que tu as appris, limites. Publie une première release et enregistre une démo de 2 minutes.

Critère de fin une commande relance l'évaluation, le README est publié, la démo est enregistrée.

Une ligne de CV prête à l'emploi

« Conception d'une salle de marché simulée multi-agents (LangGraph, Python 3.14) : 6 agents LLM (analystes, débat bull/bear, trader, risk manager), exécution parallèle, boucles bornées, validation humaine, garde-fous déterministes, persistance et rejouabilité, évaluation automatisée et traçabilité (LangSmith). »

Ajoute un chiffre mesuré dès que tu l'as : taux de sorties valides, coût par cycle, nombre de scénarios d'évaluation.

5. Règles de simulation

5.1 Conventions

Paramètre	Valeur	Pourquoi
Décision	Vendredi, après la clôture	Toutes les données de la semaine sont connues.
Exécution	Lundi, à l'ouverture	On n'exécute jamais au prix qui a servi à décider.
Slippage et frais	5 points de base ; 0,05 % du montant	Rendre chaque échange coûteux, comme en réalité.
Positions	Achats uniquement	Garder un registre simple.
Univers	3 actions, le SPY en repère	Assez pour un débat, assez peu pour tout lire.
Capital	100 000 $ fictifs	Un ordre de grandeur réaliste.

5.2 Deux biais à connaître

- **Look-ahead** : une donnée inconnue à la date de décision se glisse dans l'analyse. Parade : `snapshot(as_of)` et ses tests.
- **Mémoire du modèle** : un LLM a été entraîné sur des données passées et peut « savoir » ce qui s'est produit pendant la période rejouée. Parade : considère le mode replay comme un banc d'essai technique, et le mode prospectif comme le seul test honnête de la qualité des décisions. Le documenter dans ton README est en soi une preuve de maturité.

6. Budget

6.1 Tarifs au 16 octobre 2026

Fournisseur	Modèle	Entrée ($/M tokens)	Sortie ($/M tokens)	Usage conseillé
Anthropic	Claude Haiku 5.5	0,10 (prompt ≤ 100 k tokens)	0,50	Tous les agents
Anthropic	Claude Sonnet 5.5	2	10	Trader, pour l'expérience comparative
OpenAI	GPT-6 Luna	0,10	0,50	Alternative au même prix
Mistral	Mistral Small 4	≈ 0,15	≈ 0,60	Alternative européenne (tarif à confirmer)
Ollama	Modèles ouverts locaux	0	0	Option zéro euro (machine puissante requise)

6.2 Hypothèses de consommation

Agent (par titre et par cycle)	Appels	Tokens en entrée	Tokens en sortie
Analyste technique (2 tours d'outils)	2	5 000 par appel	600 par appel
Analyste news & macro (2 tours d'outils)	2	5 000 par appel	600 par appel
Chercheur Bull	1	5 000	500

Agent (par titre et par cycle)	Appels	Tokens en entrée	Tokens en sortie
Chercheur Bear	1	5 500	500
Trader	1	6 000	500
Risk Manager	1	4 000	300
Total par cycle (3 titres)	**24**	**≈ 121 500**	**≈ 12 600**

Estimations à ± 50 % : mesure tes vrais volumes dès la phase 4 et remplace ces hypothèses.

6.3 Coût selon la configuration

Configuration	1 cycle	1 an de replay (52 cycles)	8 semaines de prospectif
Tout en Claude Haiku 5.5 ou GPT-6 Luna	0,018 $	0,96 $	0,15 $
Tout en Mistral Small 4	0,026 $	1,34 $	0,21 $
Haiku 5.5 partout, Trader en Sonnet 5.5	0,067 $	3,48 $	0,54 $
Tout en Sonnet 5.5 (pour comparaison)	0,37 $	19,19 $	2,95 $
100 % local (Ollama)	0 $	0 $	0 $

6.4 Budget total du projet

Poste	Zéro	Économe	Recommandé
Développement et débogage	0 $	2 $	3 $
Simulations replay	0 $	4 $: 4 replays d'un an en Haiku	9 $: 2 replays en Haiku, 2 avec le Trader en Sonnet
Évaluations (15 scénarios)	0 $	0,5 $	2 $ (juge en Sonnet 5.5)
Mode prospectif, 8 semaines	0 $	0,2 $	0,5 $
Total	**0 $**	**≈ 7 $**	**≈ 15 $**
En euros (1 $ ≈ 0,87 €)	**0 €**	**≈ 6 €**	**≈ 13 €**

Aucun abonnement n'est nécessaire : les offres gratuites de LangSmith, Finnhub et FRED couvrent tout le projet. La configuration « Zéro » suppose une machine capable de faire tourner un modèle local correct (idéalement 16 Go de mémoire vidéo ou plus).

6.5 Garder la main sur les coûts

- Plafond de dépenses dans la console du fournisseur, dès le premier jour.
- `max_tokens` sur chaque modèle et `recursion_limit` sur le graphe.
- Des snapshots courts : Haiku 5.5 coûte cinq fois plus cher au-delà de 100 000 tokens de prompt.
- Le prompt caching sur les instructions système, identiques d'un appel à l'autre.
- Un faux modèle (`GenericFakeChatModel`) dans les tests : aucun appel payant en CI.

7. Planning

Environ 60 heures sur 8 semaines, à raison de deux ou trois séances par semaine. Les bonus viennent ensuite, à ton rythme.

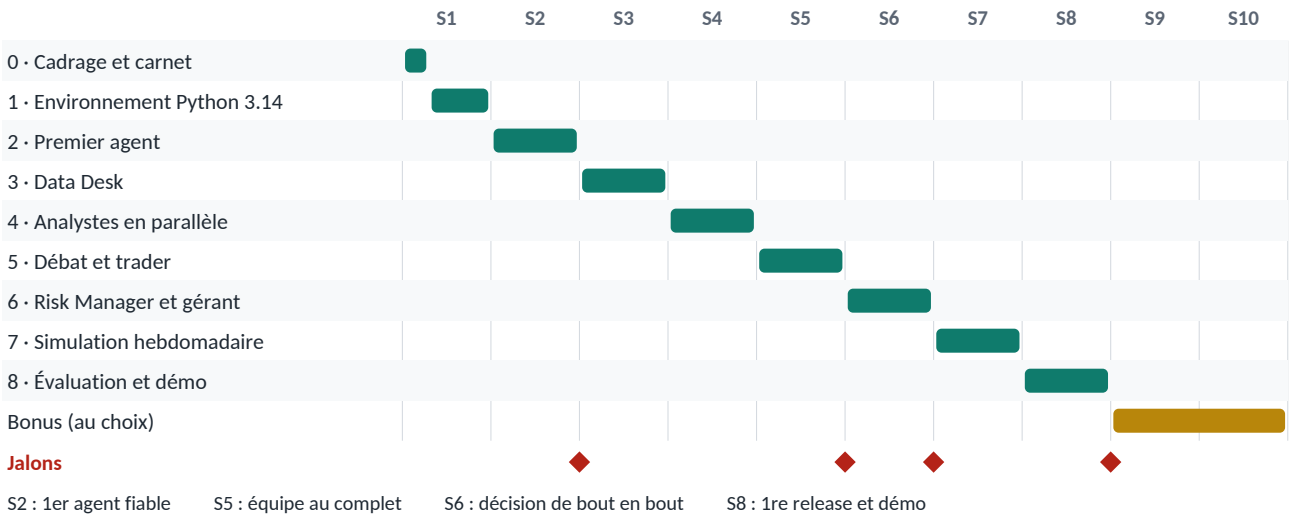


Figure 2. Planning indicatif à 7 ou 8 heures par semaine.

Semaine	Phase	Heures	Agents ajoutés	Ce que tu as à la fin
S1	0 et 1 · Cadrage, environnement	≈ 6 h	—	Cadrage, carnet, Hello LangGraph tracé
S2	2 · Premier agent	≈ 8 h	Analyste technique	Un agent outillé fiable
S3	3 · Data Desk	≈ 6 h	—	Données en cache, snapshot testé
S4	4 · Analystes en parallèle	≈ 6 h	Analyste news & macro	6 rapports par cycle
S5	5 · Débat et trader	≈ 8 h	Bull, Bear, Trader	Décisions argumentées
S6	6 · Risk Manager et gérant	≈ 7 h	Risk Manager	Un cycle complet de bout en bout
S7	7 · Simulation	≈ 8 h	—	Rapport de replay, mode prospectif lancé
S8	8 · Évaluation et démo	≈ 9 h	—	Évaluation, dashboard, README, 1re release

8. Pièges courants

Piège	Signe	Parade
Chiffres inventés	Un analyste cite un ratio absent du snapshot.	Consigne explicite et évaluateur de fidélité.
Sorties mal formées	La validation Pydantic échoue.	Sortie structurée, schéma simple, test « 10 sur 10 ».
Boucles sans fin	Le débat ou le renvoi ne s'arrête pas.	Compteurs dans l'état et recursion_limit.
Données instables	Erreurs 429, trous dans l'historique.	Téléchargement groupé et cache Parquet.
Non-déterminisme	Deux exécutions, deux décisions.	Température basse, trois exécutions, comparer.
Fuite de clés	Une clé API sur GitHub.	.env ignoré, alertes de secrets GitHub activées.
Sur-ingénierie	Deux semaines sans rien qui tourne.	Un agent et un titre d'abord ; un jalon par semaine.
Chasse au rendement	Tu retouches les prompts pour améliorer la courbe.	Revenir aux indicateurs du § 1.2 : ce sont eux qui comptent.

9. Bonus, une fois ta première release publiée

- **Mémoire long terme** : un Store LangGraph où le Trader relit ses erreurs passées avant de décider.
- **Serveur MCP** : exposer ton Data Desk comme serveur d'outils MCP et le brancher sur tes agents avec `langchain-mcp-adapters`.
- **Variante superviseur** : un agent routeur décide de l'enchaînement ; compare-la au graphe fixe sur ta batterie d'évaluation.
- **Local ou API** : même batterie, modèle Ollama contre Haiku 5.5 ; qualité, coût, latence.
- **Industrialisation** : Docker Compose, Postgres comme checkpointer, Prefect pour le cycle du vendredi. Tu connais ces outils, l'intérêt est de les appliquer à un système agentique.
- **Paper trading** : brancher les ordres validés sur un compte Alpaca de démonstration, gratuit.
- **Lecture comparative** : étudier TradingAgents (Tauric Research), un framework open source de trading multi-agents construit sur LangGraph, et noter ce que tu ferais différemment. À lire après ta première release, pas avant.

10. Ressources et sources

Ressource	Lien
Documentation LangGraph	https://docs.langchain.com/oss/python/langgraph/overview
LangChain Academy	https://academy.langchain.com
LangGraph sur PyPI	https://pypi.org/project/langgraph/
langchain-core sur PyPI (Python 3.14 déclaré)	https://pypi.org/project/langchain-core/
langchain-anthropic sur PyPI	https://pypi.org/project/langchain-anthropic/
Suivi du support Python 3.14 par LangChain	https://github.com/langchain-ai/langchain/issues/34441
Tarifs de l'API Claude	https://platform.claude.com/docs/en/about-claude/pricing
Tarifs de l'API OpenAI	https://developers.openai.com/api/docs/pricing
Tarifs LangSmith	https://www.langchain.com/pricing-langsmith
Finnhub	https://finnhub.io
FRED, documentation de l'API	https://fred.stlouisfed.org/docs/api/fred/
uv, documentation	https://docs.astral.sh/uv/
TradingAgents (Tauric Research)	https://github.com/TauricResearch/TradingAgents

Annexe A · Squelette du graphe

Ce squelette a été exécuté sous Python 3.14.6 avec des nœuds factices. Les fonctions `load_market`, `bull`, `bear`, `trader`, `risk_check`, `execute`, `book_and_report` et `ANALYST_PROMPTS` sont à écrire au fil des phases.

Squelette testé sous Python 3.14.6

```
# src/jp_morgraph/graph.py
import sqlite3
from typing import Literal
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command, Send, interrupt
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.checkpoint.serde.jsonplus import JsonPlusSerializer
from jp_morgraph.state import DeskState, AnalystReport, OrderProposal, RiskVerdict

llm = init_chat_model("anthropic:claude-haiku-5-5", temperature=0.2, max_tokens=800)
MAX_ROUNDS, MAX_RETRIES = 1, 2
KINDS = ("technique", "news_macro")

def route_to_analysts(state: DeskState) -> list[Send]:
    # un appel du nœud "analyst" par couple (titre, spécialité) : 3 x 2 = 6
    return [Send("analyst", {"ticker": t, "kind": k, "as_of": state["as_of"],
                             "features": state["snapshot"][t]})
            for t in state["universe"] for k in KINDS]

def analyst(task: dict) -> dict:
    prompt = ANALYST_PROMPTS[task["kind"]].format(**task)
    report = llm.with_structured_output(AnalystReport).invoke(prompt)
    return {"reports": [report]}                # fusionné par le reducer `add`
```

```
def debate_route(state: DeskState) -> Literal["bull", "trader"]:
    return "bull" if state["debate_round"] < MAX_ROUNDS else "trader"

def risk_route(state: DeskState) -> Literal["pm_approval", "trader", "book"]:
    if state["risk"].approved:
        return "pm_approval"
    return "trader" if state["risk_retries"] < MAX_RETRIES else "book"

def pm_approval(state: DeskState) -> dict:
    if state.get("auto_approve"):                # mode replay
        return {"approved_orders": state["orders"]}
    decision = interrupt({"orders": [o.model_dump() for o in state["orders"]]})
    return {"approved_orders": state["orders"] if decision.get("approve") else []}

def size_order(conviction: float, nav: float, held_value: float, price: float,
               max_weight: float = 0.40) -> int:
    """Le code, pas le LLM, transforme une conviction en quantité."""
    return int((conviction * max_weight * nav - held_value) // price)

builder = StateGraph(DeskState)
builder.add_node("load_market", load_market)     # Data Desk : snapshot(as_of), compteurs à 0
builder.add_node("analyst", analyst)
builder.add_node("bull", bull)
builder.add_node("bear", bear)                   # incrémente debate_round
builder.add_node("trader", trader)               # sens et conviction (LLM), taille (size_order)
builder.add_node("risk", risk_check)             # règles en code, avis LLM, risk_retries += 1
builder.add_node("pm_approval", pm_approval)
builder.add_node("execute", execute)             # exécution lundi à l'ouverture
builder.add_node("book", book_and_report)        # registre, P&L, journal

builder.add_edge(START, "load_market")
builder.add_conditional_edges("load_market", route_to_analysts, ["analyst"])
builder.add_edge("analyst", "bull")              # attend la fin des 6 analyses
builder.add_edge("bull", "bear")
builder.add_conditional_edges("bear", debate_route)
builder.add_edge("trader", "risk")
builder.add_conditional_edges("risk", risk_route)
builder.add_edge("pm_approval", "execute")
builder.add_edge("execute", "book")
builder.add_edge("book", END)

graph = builder.compile()                        # version exportée pour Studio

if __name__ == "__main__":
    # déclarer tes classes au sérialiseur évite l'avertissement « type non enregistré »
    serde = JsonPlusSerializer(allowed_msgpack_modules=[AnalystReport, OrderProposal, RiskVerdict])
    conn = sqlite3.connect("data/checkpoints.db", check_same_thread=False)
    desk = builder.compile(checkpointer=SqliteSaver(conn, serde=serde))
    config = {"configurable": {"thread_id": "replay-01/2026-10-09"}, "recursion_limit": 40}
    desk.invoke(initial_state("2026-10-09"), config)              # s'arrête sur interrupt()
    desk.invoke(Command(resume={"approve": True}), config)        # reprise après ta validation
```

Annexe B · Modèle de prompt d'analyste

```
Tu es l'analyste {kind} d'une petite salle de marché.
Mission : évaluer {ticker} au {as_of}, à partir des seules données ci-dessous.

Règles :
- N'utilise que les chiffres présents dans les données. N'en invente aucun.
- Si une information manque, écris « information insuffisante » au lieu de deviner.
- Ignore ce que tu crois savoir de l'évolution de ce titre après le {as_of}.
- Cinq arguments au maximum, du plus important au moins important.
- Réponds uniquement au format AnalystReport.

Données au {as_of} :
{features}
```

Annexe C · Checklist de la première semaine

- ☐ Installer uv, Python 3.14, Git et VS Code.
- ☐ Créer les comptes : fournisseur LLM, LangSmith, Finnhub, FRED.
- ☐ Fixer un plafond de dépenses de 5 à 10 € dans la console du fournisseur.
- ☐ Initialiser le dépôt et le .gitignore (.env, data/).
- ☐ Rédiger docs/cadrage.md et ouvrir docs/journal.md.
- ☐ Faire tourner Hello LLM et Hello LangGraph, avec une trace visible.
- ☐ Ouvrir le graphe dans LangGraph Studio.
- ☐ Réserver deux ou trois créneaux réguliers par semaine pour le projet.

Et maintenant ?

Commence par la phase 0 : deux heures pour poser le cadrage et ouvrir ton carnet. Bonne construction !